

4. COMPLEXIDADE MÉDIA

- Este capítulo considera a **complexidade média**, examinando casos ilustrativos de análise de algoritmos e de projeto de estruturas.
- Dentre as medidas de complexidade, a complexidade média é, às vezes, o critério de avaliação **mais apropriado**.
- Conforme o capítulo 2, a complexidade média de um algoritmo é uma **média ponderada** do desempenho de cada entrada pela probabilidade de ela ocorrer.
- Dado um algoritmo **a**, sejam:
 - D_n o conjunto de suas entradas com tamanho **n**,
 - $\text{prob}(d)$ a probabilidade de ocorrer a entrada **d** de **D**,
 - $\text{desemp}[\mathbf{a}](d) := \text{custo}(\text{exec}[\mathbf{a}](d))$ o desempenho de **a** com entrada **d**.
- Assim, a complexidade média do algoritmo **a**:

$$c_M[\mathbf{a}](n) := \sum_{d \in D_n} [\text{prob}(d) \cdot \text{desemp}[\mathbf{a}](d)].$$

Visão geral do capítulo

- 4.1 Um exemplo clássico: Quicksort.

Analisa o algoritmo de classificação Quicksort como um exemplo clássico de complexidade média, com distribuição uniforme.

- 4.2 Árvore de busca binária ótima.

Examina árvore de busca binária ótima: um exemplo de armazenamento de dados utilizando um critério que tem como objetivo minimizar a complexidade média de procura.

4.1 UM EXEMPLO CLÁSSICO: QUICKSORT

- O algoritmo *Quicksort* classifica uma lista, particionando-a, classificando suas partes e depois concatenando as partes classificadas.
- A idéia básica é escolher, de alguma maneira, um *pivô* e particionar a lista em relação ao pivô de maneira que todo elemento de uma das partes seja menor que qualquer elemento da outra parte.
- Este algoritmo *classifica* um vetor A em ordem *não decrescente*. O pivô será o maior entre os dois primeiros elementos diferentes em L, sendo zero se todos os elementos forem iguais entre si.

○ **Algoritmo 4.1.1: Quicksort.**

<u>Função</u> Quicksort ($A : V, m, p : \text{Nat}$) $\rightarrow A : V$	<i>{ordenação por concatenação}</i>
	<i>{Entrada: vetor $A[m..p]$ (a ordenar).}</i>
	<i>Saída: vetor $A : V$ (o vetor de entrada classificado em ordem ascendente).}</i>
<u>sorte</u> $V := \text{vetor } [m..p] \text{ de Elmord}$	
1. $k \leftarrow \text{ind}(\text{pivô}, A);$	<i>{ inicializa k com índice do pivô }</i>
2. <u>se</u> $k = 0$	<i>{ testa se todos os elementos são iguais entre si: 2 }</i>
3. <u>então</u> <u>retorne-saída</u> (A)	<i>{ vetor já está classificado }</i>
4. <u>senão</u>	<i>{ inicia classificação: 4 }</i>
5. Troca($A[m], A[k]$);	<i>{ troca primeiro com pivô }</i>
6. Particionamento(A);	<i>{ particiona na posição j }</i>
7. $L' \leftarrow \text{Quicksort}(A[m .. j - 1]);$	<i>{ classifica início (tamanho j - m) }</i>
8. $L'' \leftarrow \text{Quicksort}(A[j .. p]);$	<i>{ classifica final (tamanho p - j + 1) }</i>
9. $L \leftarrow \text{Concat}(L', L'');$	<i>{ concatena partes classificadas }</i>
10. <u>retorne-saída</u> (A);	<i>{ dá saída concatenada }</i>
11. <u>fim-senão</u> ;	<i>{ 4: inicia classificação }</i>
12. <u>fim-se</u> ;	<i>{ 2: k = 0? }</i>
13. <u>fim-Função</u> .	<i>{ fim do algoritmo Quicksort }</i>

- O desempenho de Quicksort com entrada **A** envolve as seguintes contribuições:
 - **escolha do pivô** e do algoritmo de Particionamento;
 - Quicksort com a **sublista da esquerda** ($A[m..j-1]$ com tamanho i , $i = m-j$);
 - Quicksort com a **sublista da direita** ($A[j..p]$ com tamanho $n - i$, $n = p-m+1$ e $i = m-j$).

- Assim, o **desempenho** de Quicksort com entrada **A** será dado por:

$$\text{desemp[Quicksort]}(A) = \text{desemp[Pivô_Partic]}(A) +$$

$$+ \text{desemp[Quicksort]}(A[m..j-1]) +$$

$$+ \text{desemp[Quicksort]}(A[j..p]).$$

onde, $\text{desemp[Pivô_Partic]}(A)$ é o desempenho da escolha do pivô e do algoritmo Particionamento.

- Vamos usar $\text{prob}(i)$ para a **probabilidade** de o pivô ser o $(i+1)^{\text{o}}$ elemento:

$$c_M [\text{ Quicksort }](n) = \sum_{i=1}^{n-1} \{ \text{prob}(i) \cdot [c_M [\text{ Pivô_Partic }](n) +$$

$$+ c_M [\text{ Quicksort }](i) + c_M [\text{ Quicksort }](n - i)] \}.$$

- Como o algoritmo **Particionamento** tem complexidade linear, temos a **complexidade média** $c_M [\text{Pivô_Partic}](n)$ **de ordem $O(n)$** .
- Assim, abreviando $c_M [\text{Quicksort}](n)$ por $T(n)$, a expressão anterior se torna:

$$T(n) = O\left(\sum_{i=1}^{n-1} \{ \text{prob}(i) \cdot [n + T(i) + T(n - i)] \} \right).$$

- Agora, vamos calcular a **probabilidade** $\text{prob}(i)$ de o pivô ser o $(i+1)^{\circ}$ elemento.
- A probabilidade de qualquer elemento $(1^{\circ}, 2^{\circ}, 3^{\circ}, \dots, n^{\circ})$ estar na primeira posição é $\frac{1}{n}$, e a probabilidade de o segundo elemento ser menor que ele é $\frac{i}{n-1}$.
- Assim, a probabilidade de o pivô aparecer na primeira posição e ser o $(i+1)^{\circ}$ elemento será $\frac{1}{n} \cdot \frac{i}{n-1}$.
- De forma análoga, a probabilidade de o pivô estar na segunda posição e ser o $(i+1)^{\circ}$ elemento será $\frac{1}{n} \cdot \frac{i}{n-1}$.

- Logo, a probabilidade de o pivô ser o $(i + 1)^{\text{o}}$ elemento é $\frac{2i}{n \cdot (n-1)}$, e

$$\text{prob}(i) = \frac{2i}{n \cdot (n-1)}.$$

- Assim, para $n > 1$, teremos:

$$T(n) = O\left(\sum_{i=1}^{n-1} \left(\frac{2i}{n \cdot (n-1)}\right) \cdot \left\{ \sum_{i=1}^{n-1} i \cdot [n + T(i) + T(n-1)] \right\}\right).$$

- Realmente, pode-se mostrar, por indução sobre $n > 1$, que:

$$\frac{2i}{(n-1)} \cdot \sum_{i=1}^{n-1} T(i) = O(n \cdot \log n).$$

- Portanto, podemos concluir que:

$$c_M [\text{Quicksort}](n) = O(\log n).$$

4.2 ÁRVORE DE BUSCA BINÁRIA ÓTIMA

- Agora vamos examinar a **árvore de busca binária ótima**, como um exemplo de armazenamento de dados.
- Procura-se uma estrutura de dados que privilegie os dados mais usados, que armazene, nas posições mais próximas da raiz, os **dados mais procurados**.
- A estrutura é uma **árvore binária** e a procura é a partir da raiz, usando a estratégia da busca binária. Isto é, para qualquer nodo: “os elementos da subárvore da esquerda são menores que os elementos da subárvore da direita”.
- A questão é: dada uma seqüência ordenada de elementos, com **probabilidades de procura** conhecidas, obter uma tal árvore de busca binária ótima.

- Para isso, precisamos estabelecer a “**raiz**” da árvore. Conhecida a raiz, sabe-se que os elementos **menores** que a raiz ficarão na subárvore da **esquerda**, e os **maiores**, na subárvore da **direita**.
- Seja $S = \langle x_1, x_2, \dots, x_n \rangle$ uma seqüência ordenada de n elementos a serem alocados.

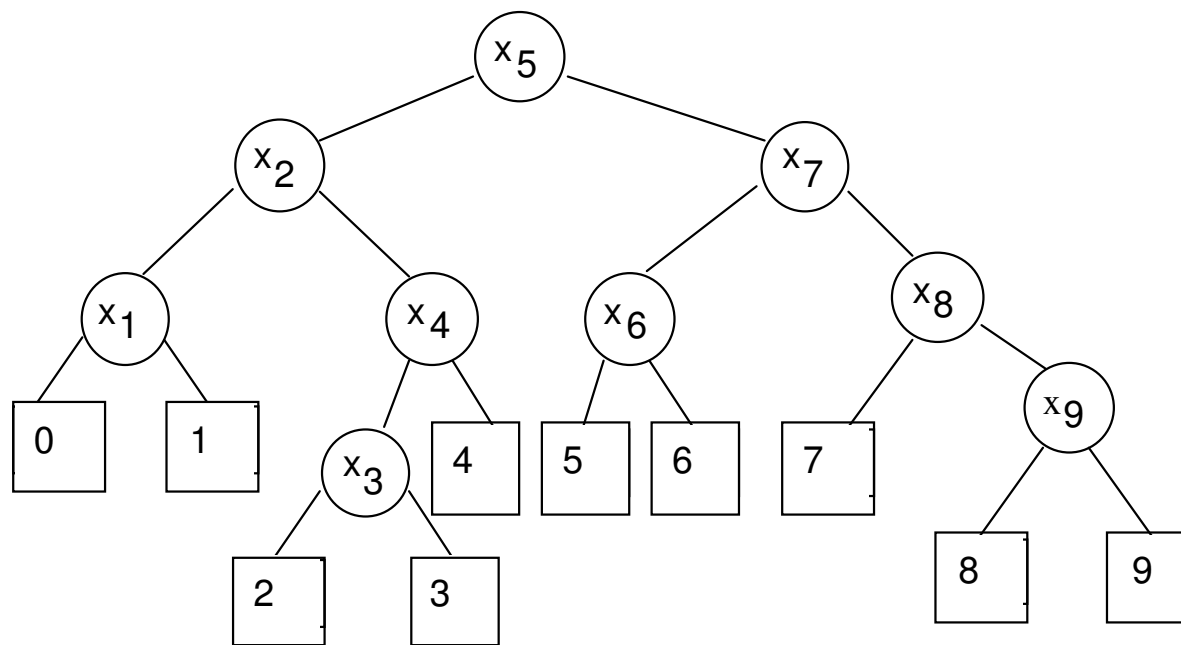


Figura 4.2.1: Árvore de busca binária

- A estrutura de dados é uma árvore de busca binária com nodos **redondos** representando elementos **presentes**, e nodos **quadrados**, elementos **ausentes**.
- Para a árvore representada na figura 4.2.1, tem-se os seguintes custos de procura c:

$$\left. \begin{array}{l} c(x_5) = 1 \\ c(x_2) = c(x_7) = 2 \\ c(x_1) = c(x_4) = c(x_6) = c(x_8) = 3 \\ c(x_3) = c(x_9) = 4 \end{array} \right\} \text{ i. e, } c(x_i) = \text{prof}(x_i) + 1$$

$$\left. \begin{array}{l} c(0) = c(1) = c(4) = c(5) = c(6) = c(7) = 3 \\ c(2) = c(3) = c(8) = c(9) = 4 \end{array} \right\} \text{ i.e, } c(i) = \text{prof}(i)$$

onde prof dá a **profundidade** do nodo na árvore.

- A complexidade média de procura na árvore é:

$$\sum_{x \in S} [\text{prob}(x) \cdot c(x)] + \sum_{x \notin S} [\text{prob}(x) \cdot c(x)]$$

onde $\text{prob}(x)$ é a **probabilidade** de procura de um elemento x e $c(x)$ dá o número de comparações necessárias para achar x ou determinar que $x \notin S$.

- Assim a complexidade média de procura em uma árvore A é dada pela seguinte equação:

$$c_M(A) = \sum_{i=1}^n (\text{prob}(x_i) \cdot \text{prof}_A(x_i)) + \underbrace{\sum_{i=1}^n \text{prob}(x_i)}_{\text{independente da árvore}} + \sum_{i=0}^n (\text{prob}(i) \cdot \text{prof}_A(i))$$

Custo (A)

- Deseja-se obter A que **minimize** $\text{Custo}(A)$:

$$\begin{aligned} \text{Custo}(A) = & \sum_{i=1}^{r-1} (\text{prob}(x_i) \cdot \text{prof}_A(x_i)) + \sum_{i=0}^n (\text{prob}(i) \cdot \text{prof}_A(i)) + \\ & + \text{prob}(x_r) \cdot \text{prof}_A(x_r) + \sum_{i=r+1}^n (\text{prob}(x_i) \cdot \text{prof}_A(x_i)) + \sum_{i=r}^n (\text{prob}(i) \cdot \text{prof}_A(i)) \end{aligned}$$

Sendo x_r a raiz de A , $\text{prof}_A(x_r) = 0$ e:

$$\begin{aligned} \text{Custo}(A) = & \sum_{i=1}^{r-1} \{\text{prob}(x_i) \cdot (\text{prof}_{AE}(x_i) + 1)\} + \sum_{i=0}^{r-1} \{\text{prob}(i) \cdot (\text{prof}_{AE}(i) + 1)\} + \\ & + \sum_{i=r+1}^n \{\text{prob}(x_i) \cdot (\text{prof}_{AD}(x_i) + 1)\} + \sum_{i=r}^n \{\text{prob}(i) \cdot (\text{prof}_{AD}(i) + 1)\} \quad (4.3.3) \end{aligned}$$

- Conhecida a raiz de A, por exemplo x_r :

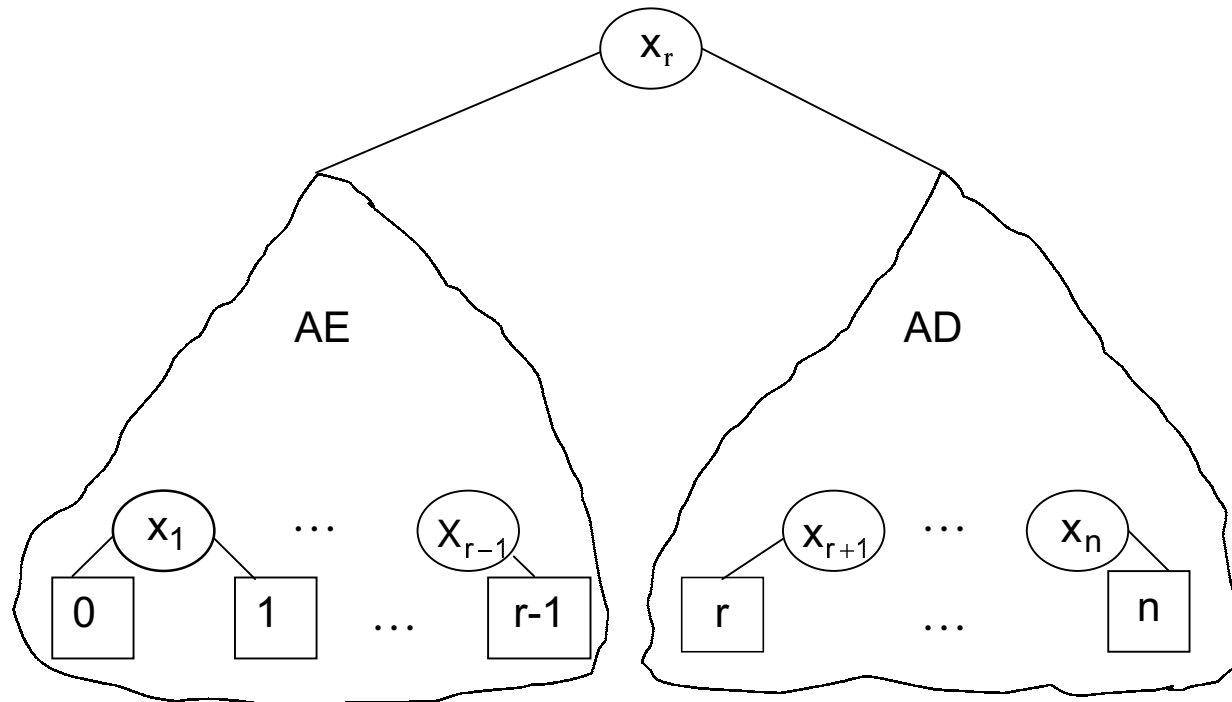


Figura 4.2.2: Subárvore da esquerda e da direita

- **AE** é a subárvore da esquerda
- **AD** é a subárvore da direita

- Temos os **custos** das subárvores dados por:

$$\text{Custo(AE)} = \sum_{i=1}^{r-1} \{\text{prob}(x_i) \cdot (\text{prof}_{\text{AE}}(x_i) + 1)\} + \sum_{i=0}^{r-1} \{\text{prob}(i) \cdot (\text{prof}_{\text{AE}}(i) + 1)\},$$

$$\text{Custo(AD)} = \sum_{i=r+1}^n (\text{prob}(x_i) \cdot \text{prof}_A(x_i)) + \sum_{i=r}^n (\text{prob}(i) \cdot \text{prof}_A(i))$$

- Chamemos:

$$\text{Prob(AE)} := \sum_{i=1}^{r-1} \text{prob}(x_i) + \sum_{i=0}^{r-1} \text{prob}(i),$$

$$\text{Prob(AD)} := \sum_{i=r+1}^n \text{prob}(x_i) + \sum_{i=r}^n \text{prob}(i)$$

- A equação 4.3.3 é reescrita como:

$$\text{Custo (A)} = \text{Custo(AE)} + \text{Prob(AE)} + \text{Custo(AD)} + \text{Prob(AD)}.$$

- Assim, o problema para $S = \langle x_1, x_2, \dots, x_n \rangle$, a partir de uma escolha de raiz x_r , passa a ter **dois subproblemas**, para:
 $S_1 = \langle x_1, x_2, \dots, x_{r-1} \rangle$ e $S_2 = \langle x_{r+1}, x_{r+2}, \dots, x_n \rangle$.
- A questão então é a escolha adequada de r .
- Esse problema será resolvido construindo a solução a partir de um **subcaso** para o caso geral, partindo do caso inicial com a seqüência S vazia.
- Construção da solução passo a passo:

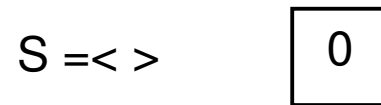


Figura 4.2.3: Árvore inicial

- Em um segundo passo:

$$S_1 = \langle x_1 \rangle, S_2 = \langle x_2 \rangle, \dots, S_n = \langle x_n \rangle;$$

são n seqüências de um elemento e serão obtidas n árvores, conforme a figura 4.2.4.

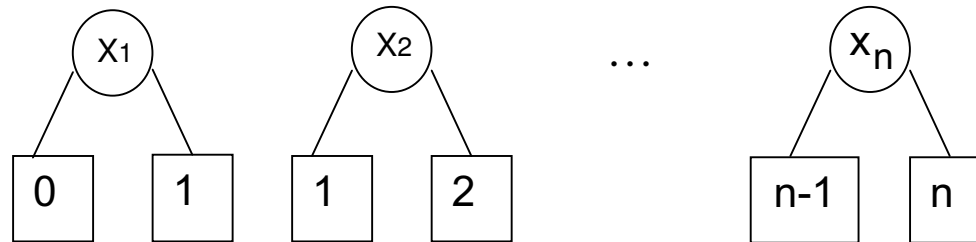


Figura 4.2.4: Primeiro nível de subárvores

- No próximo passo, consideramos seqüências com dois elementos:
 $S_1 = \langle x_1, x_2 \rangle$, $S_2 = \langle x_2, x_3 \rangle$, ..., $S_{n-1} = \langle x_{n-1}, x_n \rangle$.
- Para $S_1 = \langle x_1, x_2 \rangle$, há duas possibilidades, como mostra a figura 4.2.5.

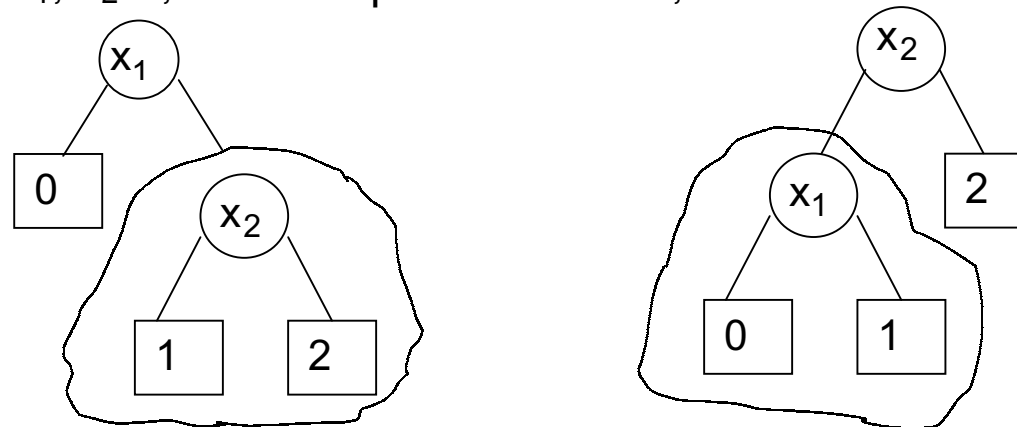


Figura 4.2.5: Subárvore de segundo nível

A de **menor custo** é escolhida.

- Assim, chegamos ao algoritmo a seguir.
- *Algoritmo 4.2.1: **Árvore de busca binária ótima***
- Este algoritmo determina uma árvore de busca binária de complexidade média ótima. Ele usa as probabilidades de procura dos elementos que estão e que não estão na árvore, e fornece as raízes das árvores ótimas em uma matriz.
- O algoritmo determina a melhor raiz, calcula os custos e a soma das probabilidades das subárvores menores (de altura s), guardando-os em raiz, em custo e em P , para utilizar nas próximas iterações. No final, o índice correspondente à raiz da árvore estará em $\text{raiz}[1, n]$ e, a partir daí, a árvore inteira pode ser recuperada.

Procedimento Árvore_Busca_Binária_Ótima { *árvore de busca binária ótima* }

 { **Entrada**: Prob (probab. de procura dos elementos dentro ou fora da árvore).

Saída: matriz raiz : M ($\text{raiz}[i, j]$ é raiz de árvore ótima para $\langle x_i, \dots, x_j \rangle$). }

sorte $M := \text{vetor} [1..n, 1..n]$ de $\mathbb{R}_+$;

var {auxiliar} $P, S : M$ {matrizes}; k {índice};

1. <u>para</u> i <u>de</u> 1 <u>até</u> n <u>faça</u>	<i>{ inicializações }</i>
2. $\text{custo}[i, i] \leftarrow 0;$	
3. $P[i, i] \leftarrow \text{Prob}[i];$	
4. <u>fim-para</u> ;	<i>{ 1: i de 1 até n }</i>
5. <u>para</u> s <u>de</u> 1 <u>até</u> n <u>faça</u>	
6. <u>para</u> i <u>de</u> 0 <u>até</u> $n - s$ <u>faça</u>	
7. $j \leftarrow i + s;$	
8. <u>com</u> $S[i, k, j] := \text{custo}[i, k-1] + P[i, k-1] + \text{custo}[k, j] + P[k, j]$ ($i < k \leq j$)	
9. determine k tal que $S[i, k, j]$ seja mínimo para $i < k \leq j$;	
10. $\text{raiz}[i, j] \leftarrow k;$	
11. $\text{custo}[i, j] \leftarrow S[i, j, k];$	
12. <u>fim-com</u>	<i>{ 8: $S[i, k, j]$, para $i < k \leq j$ }</i>
13. $P[i, j] \leftarrow P[i, j - 1] + \text{Prob}[i] + \text{Prob}[j];$	
14. <u>fim-para</u> ;	<i>{ 6: i de 0 até $n - s$ }</i>
15. <u>fim-para</u> ;	<i>{ 5: s de 1 até n }</i>
16. <u>pare-com-saída</u> (custo , raiz);	<i>{ dá a saída }</i>
17. <u>fim-Procedimento</u> .	<i>{ fim de Árvore_Busca_Binária_Ótima }</i>

4.3 RESUMO DO CAPÍTULO

- A finalidade deste capítulo é examinar a complexidade média, considerando casos ilustrativos de análise de algoritmos e de projeto de estruturas.
- Na seção 4.1, foi analisado o algoritmo de classificação Quicksort, como um exemplo clássico de complexidade média com distribuição uniforme. O desempenho desse algoritmo depende crucialmente de qual elemento vem a ser o pivô, por isso teve que ser determinada a probabilidade de posição do pivô.
- Mostrou-se que sua complexidade média é de ordem $O(n \cdot \log n)$. Esse valor é ótimo para algoritmos de classificação, conforme será visto mais tarde.
- A seção 4.2 examinou árvore de busca binária ótima como um exemplo de armazenamento de dados, utilizando um critério que tem como objetivo minimizar a complexidade média de procura.
- A partir de uma análise da influência de cada nodo na complexidade média obteve-se o algoritmo 4.2.1: Árvore de Busca Binária Ótima, que determina uma árvore de busca binária com complexidade média ótima.